

Portafolio Personal

Documentación técnica

Luisana Jacqueline Gutiérrez Ruggia

Mayo 2026

Resumen

Este documento describe la arquitectura, las tecnologías y las decisiones de diseño detrás del Portafolio Personal de Luisana Gutiérrez Ruggia. El portafolio es una single page application estática construida con React y TypeScript que sirve como vitrina interactiva de los proyectos de la autora en ciencia e ingeniería de datos. El sitio combina elementos visuales tridimensionales, un grafo conceptual force-directed estilo Obsidian, un asistente conversacional con modelo de IA y soporte completo bilingüe para visitantes de habla hispana e inglesa.

El proyecto resuelve dos problemas concretos. Primero, ofrece una alternativa a los portafolios estáticos planos que dominan la industria al introducir interactividad significativa que invita al visitante a explorar el contenido en lugar de simplemente leerlo. Segundo, demuestra capacidad técnica de la autora más allá del título de cada proyecto, exponiendo decisiones arquitectónicas, conceptos abstractos involucrados y skills aplicadas a través de visualizaciones explícitas.

El portafolio vive en GitHub Pages bajo el dominio `luisanaruggia.github.io/portfolio` y se despliega automáticamente con GitHub Actions en cada push a la rama principal.

Arquitectura

El portafolio sigue una arquitectura JAMstack pura para su versión actual, lo que significa que todo el sitio se sirve como archivos estáticos generados durante el build. No existe servidor backend en ejecución. El bundle compilado mide aproximadamente ciento ochenta kilobytes comprimidos y se sirve completo desde la red de distribución de contenido de GitHub Pages.

La aplicación es una single page application escrita en React 18 sobre TypeScript en modo estricto. El bundler y dev server es Vite, que aprovecha módulos nativos del navegador durante el desarrollo para ofrecer hot module replacement instantáneo y empaqueta con Rollup para producción. El estilo se construye con Tailwind CSS aplicando utility classes directamente en los componentes, complementado por un plugin de animaciones y un par de utilidades para componer clases condicionales.

El sistema de navegación entre vistas implementa hash routing custom sin depender de react-router. Un hook propio llamado `useHashRoute` lee `window.location.hash`, escucha el evento `hashchange` y expone un par compuesto por el identificador de proyecto activo y una función para navegar. Cuando el hash apunta a un proyecto específico mediante el patrón `#/project/<id>`, el componente raíz renderiza la vista de detalle; en cualquier otro caso muestra el grid de carpetas. Esta decisión elimina

una dependencia externa significativa y mantiene URLs compatibles que sobreviven a refrescos del navegador.

El sistema de internacionalización es igualmente propio. Un proveedor React envuelve toda la aplicación y expone un contexto con la función de traducción y un resolvedor para campos que pueden ser un string único o un objeto bilingüe con claves para español e inglés. La preferencia de idioma persiste en localStorage entre sesiones y actualiza tanto el atributo lang del documento como el título de la pestaña al cambiar.

Stack tecnológico

La capa de desarrollo y build usa pnpm como gestor de paquetes obligatorio. Un script preinstall bloquea npm y yarn para garantizar consistencia en el lockfile. TypeScript opera en modo estricto sobre todos los archivos fuente y se ejecuta tanto en el dev server para ofrecer autocompletado como en el build mediante tsc antes de delegar el bundling a Vite. El plugin oficial de Vite para React habilita la transformación de JSX y el fast refresh durante el desarrollo.

La capa de interfaz combina cuatro tecnologías visuales principales. Tailwind CSS proporciona el sistema de utility classes que da uniformidad visual y permite cambios rápidos sin abandonar el componente. La librería lucide-react expone los iconos SVG usados en headers, botones y badges; al ser tree-shakeable, solo los iconos efectivamente importados llegan al bundle final. La animación del hero usa lottie-react para reproducir un archivo JSON exportado desde After Effects que representa una ilustración animada de una persona con un computador. Finalmente, las animaciones tridimensionales de las carpetas y el grafo conceptual force-directed se implementan con CSS transforms y SVG sin librerías de animación, lo que mantiene el bundle ligero.

La capa de APIs del navegador aprovecha capacidades nativas directamente. Web Audio API con un nodo GainNode controla el volumen de la música de fondo, sorteando una limitación de Safari iOS que ignora la propiedad volume sobre elementos HTMLAudio. El almacenamiento local persiste preferencias del visitante como el idioma, el tema y el volumen de la música. Pointer Events unifica el manejo de mouse, touch y stylus para las interacciones de arrastre del grafo conceptual y del visor de diagramas, donde también se rastrean múltiples pointers simultáneos para soportar gestos de pinch zoom en dispositivos móviles. Page Visibility API pausa la música cuando la pestaña pierde el foco para no incomodar al visitante. MediaQuery API detecta si el dispositivo soporta hover real y la preferencia del sistema entre tema claro y oscuro.

El despliegue corre completamente en infraestructura gratuita de GitHub. Un workflow declarativo en YAML escucha pushes a la rama principal, instala dependencias con pnpm, ejecuta el build de Vite y publica el directorio dist como artifact de Pages. El sitio queda disponible globalmente con HTTPS automático en menos de tres minutos desde el commit inicial.

Componentes principales

El componente raíz envuelve la aplicación en un proveedor de idioma y delega el resto a FolderPortfolio, el componente que orquesta tanto la vista de home como la vista de detalle según el estado del hash routing. La vista de home presenta un header sticky con controles de descarga del CV, ajuste de volumen, control de música, selector de idioma y selector de tema, seguido de un hero con la animación Lottie y la biografía, y un grid de seis carpetas que representan las categorías del portafolio.

Cada carpeta es un componente animado tridimensional que utiliza transformaciones CSS para rotar al pasar el cursor y desplegar un abanico de hasta cinco cards de proyectos. Las cards aplican una distancia angular dinámica que se ajusta según la cantidad de proyectos en la carpeta para evitar que dos cards queden lejos cuando deberían parecer agrupadas. Al hacer click sobre una card se abre un lightbox modal que permite navegar entre los proyectos de la categoría con flechas, retornando al grid o saltando al detalle según la acción del visitante.

La vista de detalle del proyecto presenta tres áreas principales. En el hero figuran el título en color accent, una descripción multilínea, los tags de tecnologías, las categorías incluyendo las cross-disciplina y el badge de estado con indicador de color según el progreso del proyecto. En la columna derecha se renderiza el grafo conceptual force-directed que ofrece interacción de arrastre, levitación continua, filtro por grupo y leyenda dinámica con solo los grupos efectivamente presentes en el proyecto. Debajo del hero, una cuadrícula de seis tarjetas representa las secciones del proyecto: Diagramas, Documentación, README y Stack, Enlaces, Blog y Video, y Cambios recientes. Cada tarjeta abre su contenido en un modal centrado o, en el caso especial de Diagramas, en un visor fullscreen dedicado con soporte de zoom, pan y navegación entre múltiples imágenes.

El chat asistente flotante vive en la esquina inferior derecha y permanece presente tanto en la vista de home como en la de detalle. La implementación actual usa un sistema de coincidencia por palabras clave que devuelve respuestas predefinidas en el idioma activo. La planificación contempla reemplazar esta lógica por una llamada a un Cloudflare Worker que invoca un modelo de IA con contexto extraído automáticamente del catálogo de proyectos.

Setup local

El proyecto requiere pnpm como gestor de paquetes y Node 18 o superior. Tras clonar el repositorio, ejecutar `pnpm install` instala todas las dependencias respetando el lockfile. El comando `pnpm dev` arranca el dev server de Vite por defecto en el puerto 5173 con hot module replacement activo. Si el puerto está ocupado, se puede forzar otro con `pnpm dev --port 5180 --strictPort`. El comando `pnpm build` ejecuta primero el typecheck con tsc y luego empaqueta la aplicación en el directorio dist, listo para servirse estáticamente. El comando `pnpm preview` sirve dist en local para inspeccionar el bundle de producción antes de desplegar.

Las imágenes y sonidos referenciados en el código usan la utilidad `import.meta.env.BASE_URL` para prefijar correctamente el path con el subdirectorio del despliegue en GitHub Pages. En desarrollo este prefijo es vacío y en producción es la cadena `/portfolio/`. Este detalle aplica también a las rutas referenciadas en el catálogo de proyectos.

Modelo de datos

El catálogo de proyectos vive en un único archivo TypeScript en `src/data/projects.ts` que exporta un arreglo de categorías. Cada categoría tiene un título representado como clave de traducción para preservar el bilingüismo, un gradiente CSS que define el color de la carpeta correspondiente y un arreglo de proyectos. Cada proyecto puede declarar un identificador único, una imagen, un título y descripción que aceptan tanto strings como objetos bilingües, una lista de tags, un estado dentro de tres valores posibles, enlaces opcionales a repositorio y demo, una URL de documentación, un arreglo de URLs de diagramas, una URL de video, un grafo de conceptos con nodos y aristas, y una lista opcional de categorías cross-disciplina donde el proyecto debe aparecer adicionalmente.

El campo de conceptos define la estructura del grafo force-directed. Cada nodo tiene un identificador

único, una etiqueta bilingüe y pertenece a uno de cuatro grupos semánticos: arquitectura, datos, operaciones o machine learning. Cada arista conecta dos nodos por su identificador. El componente del grafo computa la posición de cada nodo mediante una simulación física que aplica fuerzas de repulsión entre pares de nodos, atracción a lo largo de las aristas y una fuerza débil hacia el centro. El layout se calcula primero con varias iteraciones de alta intensidad para obtener un estado inicial estable y luego continúa con fuerzas mucho más débiles que producen una levitación suave durante todo el tiempo que el visitante observa el grafo.

Roadmap

La planificación actual contempla la adición progresiva de una capa de backend mínima compuesta por dos componentes runtime y una suite de agentes batch. El primer componente runtime es un Cloudflare Worker que recibe las preguntas del chat, las enriquece con contexto del portafolio y delega la respuesta a un modelo de IA. El segundo componente es un worker administrativo que recibe acciones de aprobación de borradores generados automáticamente para distribución en redes sociales.

Los agentes batch corren en GitHub Actions disparados por eventos de commit, release o cron. Algunos generan contenido para la vista del portafolio como los updates recientes, los conceptos del grafo, la documentación de proyectos y las referencias a diagramas, escribiendo el resultado como archivos versionados en el repositorio. Otros gestionan el ciclo de vida del proyecto como el agente que cambia el estado al detectar un release tag o el agente que actualiza el CV cuando un proyecto pasa a estado finalizado. Finalmente, agentes de distribución generan borradores de posts para LinkedIn y artículos para Dev.to que quedan en una cola de aprobación.

El presupuesto operativo estimado para esta fase es inferior a tres dólares mensuales, dominado por el costo variable del modelo de IA. La infraestructura de Cloudflare Workers y GitHub Actions se mantiene dentro del free tier para este tipo de uso. La elección de no usar n8n ni un servidor monolítico se basa en la naturaleza impredecible del tráfico de un portafolio personal, donde un servidor en ejecución continua desperdiciaría recursos en idle.

Adicionalmente, la documentación de cada proyecto del portafolio se mantendrá inicialmente como archivos PDF generados manualmente, con la expectativa de migrar a generación automática mediante un agente que lea el repositorio del proyecto y produzca tanto el archivo markdown como el PDF compilado con el mismo template visual que este documento.

Licencia y contacto

El código fuente del portafolio se publica como repositorio público en GitHub bajo cuenta personal de la autora. Las imágenes de referencia provienen de Unsplash bajo licencia gratuita, salvo aquellas que son captura directa del propio sitio. Las fuentes y librerías de terceros mantienen sus respectivas licencias de origen.

Para consultas o colaboración, el contacto principal es a través del perfil de GitHub de la autora en LuisanaRuggia o a través del chat asistente embebido en el propio portafolio.